

Procesos

Introducción

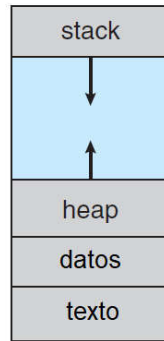
¿Qué es un proceso?

El concepto de proceso es un concepto fundamental para todo sistema operativo. Un proceso consiste de un programa en ejecución sus valores actuales y el estado y los recursos utilizados por el sistema operativo para manejar la ejecución del proceso. Un proceso es una entidad dinámica.

En un sistema operativo basado en linux en cualquier momento muchos procesos parecen estar ejecutándose concurrentemente. Por otra parte, desde el punto de vista del proceso, este tiene el acceso y control de todos los recursos del sistema como si fuera único. Ambos puntos de vista son una ilusión, la mayoría de los sistemas Linux corren en plataformas que tienen una unidad de procesamiento capaz de soportar muchos procesos activos; sin embargo, en un instante dado de tiempo solamente un proceso puede estar en ejecución. Los sistemas operativos Linux cambian rápidamente el proceso actual en ejecución, aparentando así que trabajan en forma concurrente; es decir, aparentan que trabajan en varios procesos al mismo tiempo. La capacidad de los sistemas operativos para multiplexar sus recursos entre los múltiples procesos en diferentes estados de ejecución es llamada multiprogramación (o multitarea). Los sistemas con múltiples unidades de procesamiento, las cuales por definición pueden soportar un verdadero procesamiento concurrentes se dice que tienen capacidad de multiprocesamiento.

Programa: Un programa es una entidad inactiva, estática, la cual consiste de un conjunto de instrucciones y datos asociados. Si un programa es invocado varias veces puede generar múltiples procesos.

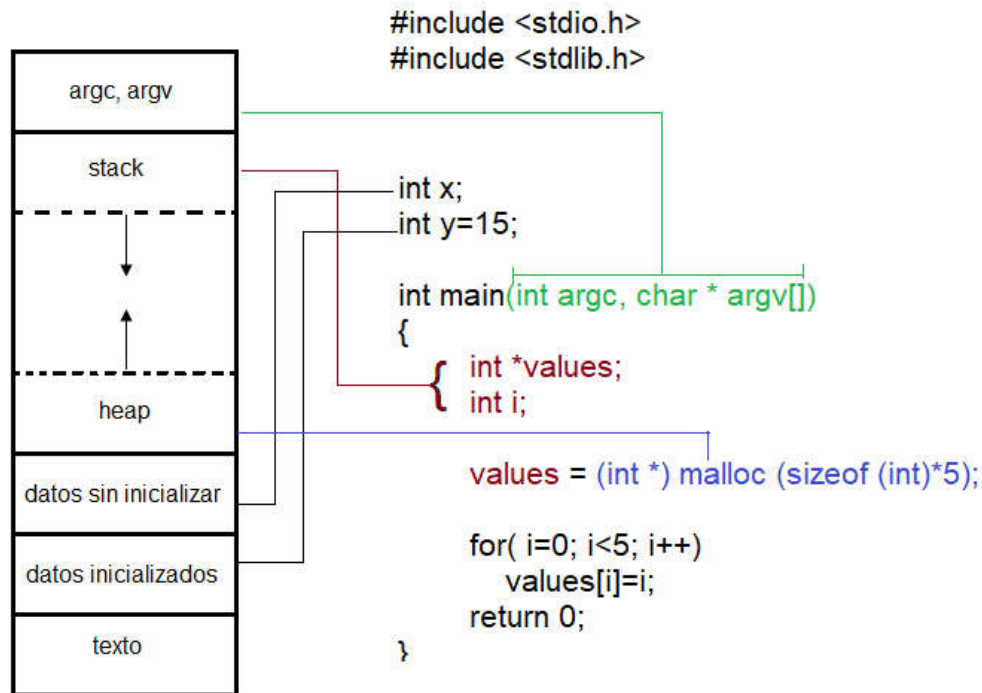
Como se mencionó, un proceso es un programa en ejecución. El estado de la actividad actual de un proceso está representado por el valor del contador del programa y el contenido de los registros del procesador. El layout de memoria de un proceso está dividido en múltiples secciones:



- Sección de texto: el código ejecutable
- Sección de datos: variables globales
- Sección Montículo o Heap: Memoria que es asignada dinámicamente durante la ejecución de un programa
- Sección Stack: Almacenamiento temporal de datos cuando se invoca a las funciones.
-

Las secciones de texto y datos tienen un tamaño fijo mientras que las secciones stack y heap pueden reducirse o crecer dinámicamente durante la ejecución del programa.

Cada vez que se llama una función se envía a la pila un registro de activación que contiene parámetros de la función, variables locales y la dirección de retorno; cuando el control regresa a la función el registro de activación se saca de la pila. Del mismo modo, la sección heap crecerá cada vez que se asigne memoria de manera dinámica y se encogerá cada vez que se libere. Aunque la pila y el montículo crecen uno hacia otro, el sistema operativo debe asegurarse que no se sobreescriban.



Procesos Importantes

Scheduler: El proceso scheduler o planificador es el algoritmo que se encarga de seleccionar a qué proceso se asigna el recurso procesador y durante cuánto tiempo. Tiene el identificador 0.

/sbin/init: es el primer proceso en ejecución tras la carga del núcleo y el que a su vez genera todos los demás procesos desde un script ubicado en `/etc/inittab`, se dice que es el padre de todos los procesos. Por lo general tiene el identificador 1.

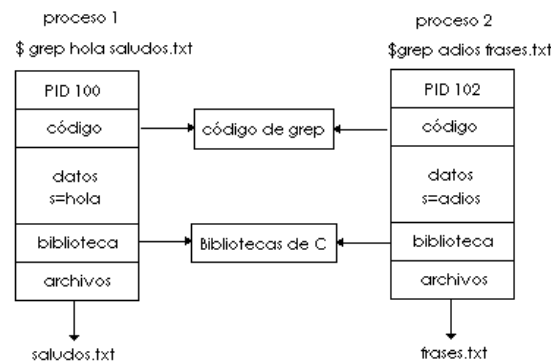
Funciones para obtener información de un proceso en C:

```
#include <sys/types.h>
#include <unistd.h>
```

<code>pid_t</code>	<code>getpid(void);</code>	retorna el identificador del proceso llamador
<code>pid_t</code>	<code>getppid(void);</code>	retorna el identificador del proceso padre
<code>uid_t</code>	<code>getuid(void);</code>	retorna el identificador de usuario real que ejecuta el programa.
<code>uid_t</code>	<code>geteuid(void);</code>	retorna el identificador de usuario efectivo. Si el programa tiene
<code>gid_t</code>	<code>getgid(void);</code>	retorna el identificador de grupo real
<code>gid_t</code>	<code>getegid(void);</code>	retorna el identificador de grupo efectivo.

Creando un proceso

Los procesos son el corazón del sistema operativo. Debe existir un mecanismo por medio del cual el sistema pueda crear un nuevo proceso. Con excepción de algunos procesos iniciales generados por el kernel durante el arranque, todos los procesos en ambiente unix/Linux son creados por un **fork()**, el proceso inicial es llamado el padre y el proceso generado es conocido como el hijo. Si tiene éxito, la llamada al sistema fork produce un proceso hijo que continua su ejecución en el punto de su invocación en el proceso padre.



Si tenemos dos procesos con números de identificación (**PID**) proceso1 y proceso2 en donde ambos utilizan el mismo comando para buscar diferentes cadenas en diferentes archivos.

Cada proceso tiene asignado un número de proceso único, conocido como identificador de proceso o **PID**. Este es usualmente un entero en el rango de 2-32000. Cuando inicia un proceso, se le asigna un número consecutivo de PID libre.

El código necesario para ejecutar el comando grep está en un archivo ordinario el cual es abierto como solo lectura con el fin de que pueda compartirse de manera segura.

Las bibliotecas también son compartidas, esto significa que solo se necesita una copia en memoria de las instrucciones, sin importar cuantos programas en ejecución podrían llamarlas.

Las variables para cada proceso son distintas, en el ejemplo anterior podemos ver que el patrón a buscar está en la variable s pero cada una está en el espacio de datos que corresponde a cada proceso, estos están separados y normalmente no pueden ser leídos por otros procesos. Los archivos de entrada que utiliza cada proceso son también diferentes, los procesos mantienen su propio juego de descriptores de archivos para su acceso.

Para implementar el modelo de procesos el sistema operativo mantiene un arreglo de estructuras llamada tabla de procesos, con una entrada por cada proceso. Cada registro contiene información acerca del estado del proceso, su contador de programa, el apuntador de pila, el reparto de memoria, la situación de sus archivos abiertos, su información de planificación y todos los demás aspectos de un proceso que se deben guardar cuando este cambia del estado de ejecutándose al estado listo, a fin de poder reiniciarlo después como si nunca se hubiera detenido.

Llamada al Sistema fork()

Fork() se utiliza para crear procesos, regresa el ID de un nuevo proceso que se dice es hijo del proceso que hizo la llamada a fork. Después que sea crea el nuevo proceso ambos procesos ejecutarán la instrucción siguiente a la llamada fork. Para diferenciar el proceso padre del hijo se usa el valor que retorna fork.

- Si fork regresa un valor negativo, significa que no se pudo crear el proceso hijo.
- Fork regresa 0 al nuevo proceso hijo
- Fork regresa un valor positivo al padre, es el ID de proceso hijo. El ID es de tipo pid_t el cual está definido en sys/types.h.
- El número de proceso también se puede obtener mediante getpid.
- Si fork falla regresará un -1 y modifica la variable global **errno**.

Ejemplo 1

```
#include<stdio.h>                                /*Generando un proceso hijo*/
#include<sys/types.h>
#include<unistd.h>
int main(void)
{
    if (fork()==0)
        printf("En el proceso HIJO\n");
    else
        printf("En el proceso PADRE\n");
    return 0;
}
```

No existe garantía de que la secuencia de salida será la generada por este programa. Si el programa es corrido numerosas veces, el enunciado "En el proceso HIJO" será desplegado antes que "En el proceso PADRE" y otras veces no ocurrirá así. La secuencia de salida es dependiente del algoritmo schedule utilizado por el kernel.

Ejemplo 2

```
#include <stdio.h>
#include <string.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#define MAX_COUNT 200
#define BUF_SIZE 100
int main(void)
{
    pid_t pid;
    int i;
    char buf[BUF_SIZE];
    fork();
    pid = getpid();
    for (i = 1; i <= MAX_COUNT; i++) {
        sprintf(buf, "Esta linea es del proceso %d,i = %d\n", pid, i);
        write(1, buf, strlen(buf));
    }
}
```

Ejemplo 3

```
#include<stdio.h>          /*Multiples actividades de los procesos PADRE--HIJO*/
#include<sys/types.h>
#include<unistd.h>
#include<string.h>
int main(void)
{ int i;
  static char buffer[10];
  if ( fork()==0)
      strcpy(buffer,"HIJO\n");          /* En el proceso HIJO */
  else
      strcpy(buffer,"PADRE\n");          /* En el proceso PADRE */
  for( i=0 ;i<5 ;++i) /*Ambos procesos hacen esto*/
  { sleep(5);
    write(1,buffer,sizeof(buffer));
  }
  return 0;
}
```

Cuando ese ejecute, la salida de este programa en un sistema local será:

```
PARENT
PARENT
CHILD
PARENT
CHILD
PARENT
CHILD
PARENT
CHILD
$ CHILD
```

La llamada **sleep** fue usada para evitar que los procesos terminen en un solo ciclo. Si existe suficiente tiempo de intervención antes de que el segundo proceso termine, el sistema desplegará el prompt produciendo la última línea de salida, donde la salida del proceso hijo se agrega al indicador de la línea de comandos (por ejemplo **\$CHILD**).

Ejemplo 4

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>

void main(void)
{
    fork();
    printf("hee\n");
    fork();
    printf("ha\n");
    fork();
    printf("ho\n");
}
```

Ejemplo 5

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#define MAX_COUNT 300
void ChildProcess(void);
void ParentProcess(void);

int main(void)
{
    pid_t pid;
    pid = fork();
    if (pid == 0)
        ChildProcess();
    else
        ParentProcess();
    return 0;
}

void ChildProcess(void)
{
    int i;
    for (i = 1; i <= MAX_COUNT; i++)
        printf("Proceso hijo, valor de i = %d\n", i);
    printf(" *** Proceso hijo terminado ***\n");
}

void ParentProcess(void)
{
    int i;
    for (i = 1; i <= MAX_COUNT; i++)
        printf("Proceso padre valor de i = %d\n", i);
    printf(" *** Proceso padre terminado ***\n");
}
```

Ejemplo 6

```
#include <sys/types.h>
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main(void)
{
    int i;
    for(i=0;i<3;i++)
    {
        if ( fork() ==0)
        {
            printf("Proceso hijo %d- %d\n", i,getpid() );
            sleep(1);
            exit(0);
        }
    }
    printf("Proceso padre % \nd", getpid() );
    return (0);
}
```

Exit

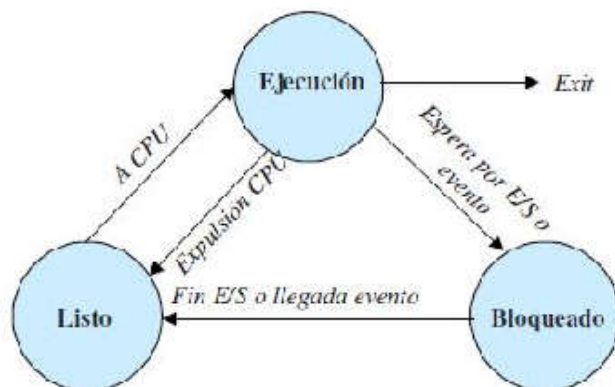
Exit, es una llamada al sistema para terminar un proceso. Liberando los recursos asignados. Se cierran todos los descriptores de archivo. Además, se cierran todos los flujos de la biblioteca de entrada/salida estándar y se vacían sus buffers, como si se hubiera llamado a la función fclose. Si los procesos hijo están aún vivos cuando exit es llamado, estos no son perturbados, pero dado que su padre está a punto de morir, su PPID se cambia a 1, el cual es el ID del proceso init.

void exit(int status)

Estado del proceso

El estado de un proceso indica la condición en que este se encuentra en un momento determinado. Muchos sistemas operativos permiten alguna forma de los estados de la siguiente tabla:

Estado	Significado
Nuevo (new)	Creación del proceso.
Ejecución (running)	Ejecución de las instrucciones del proceso.
Bloqueado (blocked)	Proceso en espera de un evento, como una operación de entrada-salida.
Listo (ready)	Proceso en espera de ser asignado al procesador.
Hecho (done)	Proceso terminado y recuperación de sus recursos.



Cambiar diagrama

El comando **ps** muestra información sobre proceso. De manera preestablecida, **ps** presenta información sobre los procesos asociados con el usuario.

Modos

- a** Muestra información de los procesos de otras terminales.
- l** Muestra toda la información disponible acerca de cada proceso.
- u** Muestra el nombre del dueño del proceso y la hora de inicio del proceso.
- F** Muestra el árbol de procesos.
- S** Muestra el tiempo de procesamiento del proceso hijo.
- e** Muestra las variables de ambiente.
- r** Muestra únicamente los procesos en ejecución

Ejemplo:

ps a u r Muestra el dueño y la hora de todos los procesos en ejecución, incluyendo otras terminales.

ps l Genera una salida como la siguiente:

```

  FLAGS  UID  PID  PPID  PRI  NI  SIZE   RSS  WCHANSTA  TTYTIMECOMMAND
  100    544  7157  7156   5   0  1228   804  wait4    S   p2  0:00 -bash
  100000  544  7428  7157   60  960   500           R   p20:00 ps -l

```

Encabezado	Significado
Flags	Banderas asociadas con el proceso.
UID	ID del usuario propietario del proceso.
PID	ID del proceso.
PPID	ID del padre del proceso.
PRI	Prioridad del proceso.
NI	Valor de nice.

ADDR	Dirección en memoria del proceso.
SIZE	Tamaño de la imagen del proceso.
WCHAN	Dirección del evento si el proceso está suspendido.
STAT	Estatus.
TTY	Terminal de control.
TIME	Tiempo de ejecución acumulado.
COMMAND	Nombre del comando.

Llamada wait

La llamada al sistema **wait** detiene al proceso que llama hasta que un hijo de este termine o se detenga, o hasta que el proceso que la invocó reciba una señal. **wait** regresa de inmediato si el proceso no tiene hijos o si el hijo termina o se detiene y aún no se ha solicitado la espera.

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t wait (int *status);
```

Ejemplo wait1.c

```
// wait1.c Prueba de wait
```

```
#include<stdio.h>
```

```
#include<sys/wait.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    if (fork()== 0)
```

```
        printf("Hola desde el hijo\n");
```

```
    else
```

```
    {
```

```
        printf("Hola desde el padre\n");
```

```
        wait(NULL);
```

```
        printf("El hijo ha terminado \n");
```

```
    }
```

```
    printf("Fin del programa\n");
```

```
    return 0;
```

```
}
```

Ejemplo 2 wait2.c

```
// wait2.c Prueba de wait
#include<stdio.h>
#include<sys/wait.h>
#include<unistd.h>
int main()
{ int i;
    if (fork()== 0)
    {        printf("Hola desde el hijo\n");
              for(i=0;i<3;i++)
              {
                  sleep(1);
                  printf("Contando %d \n", i);
              }
    }
    else
    {        printf("Hola desde el padre\n");
              wait(NULL);
              printf("El hijo ha terminado \n");
    }

    return 0;
}
```

Exit Child status

Wait puede reportar a través de macros, información acerca del estado de un hijo al terminar, esta información incluye: terminación normal o anormal, causa de terminación y estado de salida.

Macro	Valor que regresa.
WIFEXITED(stat)	Verdadero si el proceso terminó normalmente.
WEXITSTATUS(stat)	Si el proceso termina normalmente regresa el pid del proceso hijo.
WIFSIGNALED(stat)	Regresa un valor no cero si el proceso hijo termina debido a una señal .
WTERMSIG(stat)	Regresa el valor de la señal de terminación
WIFSTOPPED(stat)	Regresa verdadero su el proceso está detenido.
WSTOPSIG(stat)	Regresa la señal que provocó que el proceso se detuviera.
WIFCONTINUED(stat)	Regresa un valor no cero si el proceso ha continuado.
WCOREDUMP(stat)	Regresa verdadero si se ha generado un archivo core.

// Status Wait

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/wait.h>
#include<unistd.h>
```

Ejemplo 3 wait3.c

```
//Wait3.c
void waitEstado()
{
    int status;

    if (fork() == 0)
        exit(1);
    else
        wait(&status);
    if (WIFEXITED(status))
        printf("Estatus al salir Exit: %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        psignal(WTERMSIG(status), "Exit signal");
}

int main()
{
    waitEstado();
    return 0;
}
```

Waitpid

Aunque la señal wait es útil, tiene algunas limitaciones por ejemplo, siempre regresa el estado del primer proceso hijo que termina o se detiene. De esta manera, si la información no corresponde al proceso hijo que deseamos, será necesario guardar esta información para una posible referencia futura y realizar llamadas adicionales a wait. Waitpid, es otra función, más flexible y más compleja, capaz de solucionar estos inconvenientes.

```
#include <sys/types.h>
#include <sys/wait.h>
```

```
pid_t waitpid (pid_t pid, int *status, int options);
```

El valor de pid puede ser uno de los siguientes:

- < -1 lo que significa esperar a que cualquier proceso hijo cuyo ID del proceso es igual al valor absoluto de pid.
- 1 lo que significa que espera por cualquier proceso hijo; este es el mismo comportamiento que tiene **wait**.

0 lo que significa que espera por cualquier proceso hijo cuyo ID es igual al del proceso llamante.

> 0 lo que significa que espera por el proceso hijo cuyo ID es igual al valor de pid.

El valor de options es un OR de cero o más de las siguientes constantes:

WNOHANG

que significa que vuelve inmediatamente si ningún hijo ha terminado.

WUNTRACED

que significa que también vuelve si hay hijos detenidos, y de cuyo estado no ha recibido notificación.

El estado para los hijos rastreados que están parados también se proporciona sin esta opción.

Si status no es **NULL**, **wait** o **waitpid** almacenan la información de estado en la memoria apuntada por status.

Wait4.c

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/wait.h>
#include<unistd.h>

void waitexample()
{
    int i, stat;
    pid_t pid[5];
    for (i=0; i<5; i++)
    {
        if ((pid[i] = fork()) == 0)
        {
            sleep(1);
            exit(100 + i);
        }
    }
    .
    for (i=0; i<5; i++)
    {
        pid_t cpid = waitpid(pid[i], &stat, 0);
        if (WIFEXITED(stat))
            printf("Child %d terminated with status: %d\n",
                cpid, WEXITSTATUS(stat));
    }
}
```

```
int main()
{
waitexample();
return 0;
}
```

Llamada exec al sistema.

La llamada fork al sistema crea una copia del proceso que la llama. Muchas aplicaciones requieren que el proceso hijo ejecute un código diferente del de su padre. La familia **exec** de llamadas al sistema proporciona una característica que permite traslapar al proceso que llama con un módulo ejecutable nuevo. La manera tradicional de utilizar la combinación **fork-exec** es dejar que el hijo ejecute el **exec** para el nuevo programa mientras el padre continúa con la ejecución del código original.

Las seis variaciones de la llamada exec al sistema se distinguen por la forma en que son pasados los argumentos de la línea de comando y el ambiente, y por si es necesario proporcionar la ruta de acceso y el nombre del archivo ejecutable. Las llamadas **execl** (**execl**, **execlp** y **execle**) pasan los argumentos de la línea comando como una lista y son útiles si se conoce el número de argumentos al momento de la compilación. Las llamadas **execv** (**execv**, **execvp** y **execve**) pasan los argumentos de la línea comando en un arreglo de argumentos.

Formato

```
#include <unistd.h>
int execl (const char *path, const char *arg0, ..., const char *argn, char * /*NULL*/);
int execlp (const char *file, const char *arg0, ..., const char *argn, char * /*NULL*/);
int execle (const char *path, char *const arg0[], ..., const char *argn, char * /*NULL*/, char *const envp[]);
```

1 Execl

La función reemplaza el proceso actual con un nuevo proceso el cual está especificado en sus parámetros.

El siguiente código llama al comando ls con un argumento de línea de comando igual a -l. El programa supone que ls está localizado en el directorio /usr/bin. El parámetro path del **execl** es la ruta de acceso y el nombre del programa, especificado ya sea como nombre con la ruta completa o relativa al directorio de trabajo. Después aparecen los argumentos de la línea de comandos seguidos de un apuntador NULL. La lista de parámetros de cadena de caracteres corresponde al arreglo argv para el comando **exec**. Puesto que argv[0] es el nombre del programa, éste es el segundo argumento de **execl**.

Ejemplo execl1.c

```
// Execl1.c Con un comando del sistema operativo
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main (void)
{
    pid_t childpid;
    int status;
    if (( childpid = fork()) == -1) {
        perror ("Error al ejecutar fork");
        exit(1);
    } else if ( childpid == 0) { /* código del hijo */
        if (execl ("/usr/bin/ls", "ls", "-l", NULL) < 0 ) {
            perror ("Falla en la ejecución de ls ");
            exit(1);
        }
    } else if (childpid !=wait (&status)) /* código del padre */
        perror ("Se presentó una señal antes de la terminación del hijo");
    exit(0);
    return (0);
}
```

Ejemplo execl2.c El script debe iniciar con #!/bin/sh

```
//execl2.c Con un script
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main (void)
{
    pid_t childpid;
    int status;
    if (( childpid = fork()) == -1)
    {
        perror ("Error al ejecutar fork");
        exit(1);
    } else if ( childpid == 0)
    {
        /* código del hijo */
        if (execl ("/home/sist20232/lety/funcionesExec/holaScript", NULL) < 0 ) {
            perror ("Falla en la ejecución del script ");
            exit(1);
        }
    } else if (childpid !=wait (&status)) /* código del padre */
        perror ("Se presentó una señal antes de la terminación del hijo");
    exit(0);
    return (0);
}
```

2. execlp

Una forma alternativa es **execlp**, la cual tiene los mismos parámetros que **execl** pero utiliza la variable de ambiente PATH para buscar el ejecutable. De manera similar, cuando el usuario introduce un comando, el shell intenta localizar el archivo ejecutable en uno de los directorios especificados por la variable PATH.

Ejemplo execlp1.c

```
// Execlp llama a ambiente.c
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

int main (int argc, char *argv[])
{
    pid_t childpid;
    int status;

    if ((childpid = fork()) == -1)
    {
        perror("Error al ejecutar fork");
        exit(1);
    }
    else
    {
        if (childpid == 0) /* código del hijo */
        {
            if (execlp ("ambiente", (char *)0) < 0 )
            {
                perror ("Error en la ejecución");
                exit(1);
            }
        }
        else /* código del padre */
        {
            while(childpid != wait (&status))
                if((childpid == -1) && (errno != EINTR))
                    break;

            return(0);
        }
    }
}
```

ambiente.c

```
#include <stdio.h>

int main(int argc, char *argv[], char * envp[])
{
    int i;
    for (i = 0; envp[i] != NULL; i++)
    {
        printf("\n%s", envp[i]);
    }
    return 0;
}
```


Ejemplo execlp2.c

//Execlp2 llama al script saludoScript

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

int main (int argc, char *argv[])
{
    pid_t childpid;
    int status;

    if ((childpid = fork()) == -1)
    {
        perror("Error al ejecutar fork");
        exit(1);
    }
    else
    {
        if (childpid == 0) /* código del hijo */
        {
            if (execlp ("saludoScript", "saludoScript", "alma", "jagger", "leticia", (char *)0) < 0 )
            {
                perror ("Error en la ejecución");
                exit(1);
            }
        }
        else /* código del padre */
        {
            while(childpid != wait (&status))
                if((childpid == -1) && (errno != EINTR))
                    break;
        }
    }

    return(0);
}
```

saludoScript

```
#!/bin/sh
echo "Hola $1 $2 $3"
```

3. execl

Una tercera forma de execl es execl, la cual es similar a execl con la excepción de que toma un parámetro adicional que representa el nuevo ambiente del programa por ejecutar. En el caso de las otras formas de execl, el nuevo programa hereda el ambiente del padre.

Ejemplo execl1.c

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[], char * envp[])
{
    int i;
    pid_t pid;
    char * envp2[] = {"PERRO=Pepper", "GATO=Jagger"};

    printf ("Parametro 1 del ambiente original: %s\n", argv[1]);
    printf ("Variable GATO del ambiente 2 aun no existe: %s \n", getenv("GATO"));
    if ((pid = fork()) == -1)
        perror("fork error");
    else if (pid == 0) {
        execl("./muestraVariables", "muestraVariables", NULL, envp2);
        printf("error");
    }
}
```

muestraVariables.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{

    printf("Despues de cargar el ambiente 2 \n");
    printf("Perro: %s\n", argv[1]);
    printf("Gato: %s\n", getenv("GATO"));
    return 0;
}
```

```
#include <unistd.h>
```

```
int main(void) {  
    char *binaryPath = "/bin/bash";  
    char *arg1 = "-c";  
    char *arg2 = "echo \"Visita $HOSTNAME:$PORT.\"";  
    char *const env[] = {"HOSTNAME=fcqi.tij.uabc.mx", "PORT=8080", NULL};  
  
    execl(binaryPath, binaryPath, arg1, arg2, NULL, env);  
  
    return 0;  
}
```

Mostrar variables de ambiente

```
#include <stdio.h>  
int main(int argc, char *argv[], char * envp[])  
{  
    int i;  
    for (i = 0; envp[i] != NULL; i++)  
    {  
        printf("\n%s", envp[i]);  
    }  
    return 0;  
}
```